

Schritt-für-Schritt Plan: FreeRTOS Migration (Dukatimer v0.5)

Prämisse: Keine Code-Löschungen auf Verdacht. Erhalt aller bestehenden Funktionen. Strikt deterministisches Verhalten durch physische und logische Entkopplung der CPU-Kerne (Core 0: IO/UI, Core 1: Echtzeit-Physik).

Phase 1: Die Risiko-Inventur (Abgeschlossen für TestStrip)

Aktion: Analyse der Code-Abhängigkeiten und Identifikation potenzieller Blockaden.

- **ToDo 1:** Vollständige Sichtung von `HW_Input.cpp` und `HW_Lights.cpp` (erledigt).
- **ToDo 2:** Analyse von `Mode_TestStrip.cpp` auf Blocker (erledigt).
 - *Befund:* I2C-Aufrufe (`smartLCD`), Audio-Beeps und Software-Timing (`millis()`) blockieren aktuell den sequenziellen Ablauf.
- **ToDo 3:** Identifikation blockierender Funktionen in weiteren Modulen (Calibration, Densitometer).
- **ToDo 4: Kritischer Check:** Prüfung auf I2C-Bus-Blockaden. Hängende Hardware darf Core 1 niemals in eine Warte-Dissoziation zwingen.

Phase 2: Thread-Sicherheit & "Shadow Copies" (Das Fundament)

Aktion: Vorbereitung der Datenstruktur für den Multicore-Betrieb. Core 1 (Echtzeit) darf niemals auf Core 0 (UI) warten müssen.

- **ToDo 1:** Einführung von `InputQueue` in `Globals.h`.
- **ToDo 2: Shadow Copy Pattern für Displays:** * Core 1 schreibt Display-Texte nur in einen `struct DisplayBuffer`.
 - Core 0 liest diesen Puffer asynchron und führt die I2C-Befehle (`lcd.print`) aus.
- **ToDo 3: Audio-Queue:** * `beep...()` Funktionen senden nur noch eine `uint8_t` ID an eine `SoundQueue`.
 - Ein passiver Task auf Core 0 spielt die Töne ab. Core 1 bleibt jitterfrei.

Phase 3: Input & Hardware entkoppeln (HAL-Ebene)

Aktion: Hardware-Signale von der Logik trennen.

- **ToDo 1: Interrupt-Migration:** Umbau von `HW_Input.cpp` auf `attachInterrupt()` für alle mechanischen Taster.
- **ToDo 2: HAL-Refactoring:** `HW_Lights.cpp` wird zum reinen Befehlsempfänger. Alle `millis()` -Hüllkurven werden entfernt.
- **ToDo 3: State Verification:** Implementierung einer Rückmeldung für kritische Schaltungen (z.B. Relais-Bestätigung).

Phase 4: Die Spaltung der `main.cpp` (Operation am offenen Herzen)

Aktion: Physische Trennung der Aufgaben.

- **ToDo 1: TaskIO (Core 0, Priorität 1):** * Nextion UART, LCD-Update (via Shadow Buffer), Sensoren, Audio-Task.
 - Läuft mit "Best Effort" – Latenzen sind hier unkritisch.
- **ToDo 2: TaskRealtime (Core 1, Priorität 10 - HOCH):** * Closed-Loop Regelung und Hardware-Timer.
 - Reagiert ausschließlich auf die `InputQueue` .
 - **Predictive Shutoff:** Ersatz von `if (now >= end)` durch programmierte Hardware-Abschaltung.

Phase 5: Integration des Hardware-Schalters & Mode-Guard

Aktion: Aktivierung der physikalischen "Zeit/Dosis"-Weiche.

- **ToDo 1:** Schalter in `HW_Input.cpp` via ISR einbinden.
- **ToDo 2: Mode-Guard:** Sperre für den Start-Taster im Dosis-Modus, falls kein valider Sensor-Referenzwert vorliegt.
- **ToDo 3:** Akustisches Profiling des Modus-Wechsels.